Haixiao Wang

Week 8
Unconstrained Optimal control

16.04.2026

Motivation for MPC

↳ Till now, we learned many classical control methods (PID or standard LQR)

↳ Useful, but we can't implement any limitations / constraints effectively

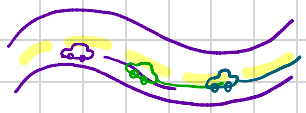
↳ Example:

↳ A steering wheel can't turn infinitely

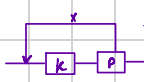
↳ A road has limits which the car can't exceed

↳ MPC uses an optimization based algorithm to anticipate the future!

Receding Horizon

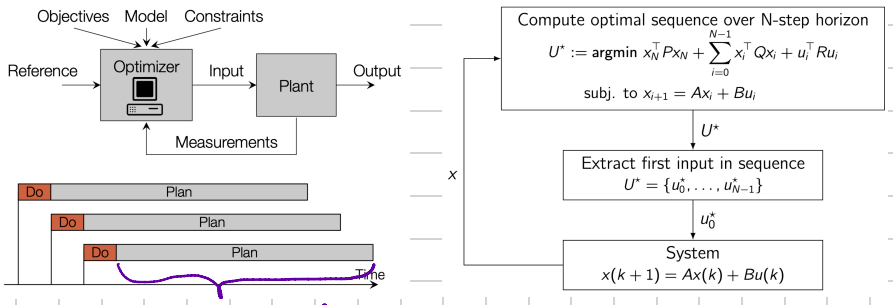


↳ Up until now, our system only looks at its present state



↳ LQR, feedback of current state!

↳



predictions N steps into future

Cost function

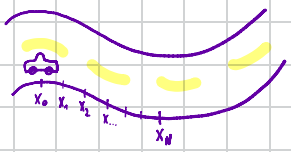
↳ Now, let's look into the future!

↳ Given we have a DT system $x(k+1) = Ax(k) + Bu(k)$

↳ To decide, what our best move ist, we penalize "worse moves" with a cost function J

↳ $J(x_0, U) = x_N^T P x_N + \sum_{i=0}^{N-1} (x_i^T Q x_i + u_i^T R u_i)$ ⇝ We want to minimize J!

↳ Example:



↳ Q keeps the car on the road, punishing when state doesn't match reference

↳ R punishes inefficiently slamming the brakes / gas

↳ P makes sure the system is aware that there is more after x_N (ensure long term stability)
(Usually solve $P = A^TPA + Q$

↳ x_0 is assumed to be given as a current state, where $x_1, x_2 \ldots x_N$ are predictions and u_i are optimization variables

↳ Now, how do we actually find these predictions?

Batch approach

↳ We use u_i and x_0 to "calculate" all future states x_i

$$X = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ \vdots \\ x_N \end{bmatrix} = \begin{bmatrix} I \\ A \\ \vdots \\ \vdots \\ A^N \end{bmatrix} x(0) + \begin{bmatrix} 0 & \cdots & \cdots & 0 \\ B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & \ddots & \ddots & 0 \\ A^{N-1}B & \cdots & AB & B \end{bmatrix} \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ \vdots \\ u_{N-1} \end{bmatrix}$$
$$= S^x x(0) + S^u U$$

↳ So we now know all future states up to x_N precisely depending on u!

↳ We insert $X = S^x x(0) + S^u U$ back into the cost function

↳ $J(x(0), U) = X^T \bar{Q} X + U^T \bar{R} U$ $\quad \bar{Q} = \text{diag}(Q, \ldots Q, P)$ $\quad \bar{R} = \text{diag}(R \ldots R)$

↳ Now, $J(x(0), U)$ is positive definite, so we take it's derivative, to find the optimal u!

↳ $\nabla_U J(x(0), U) = 0$ ⇝ delivers our desired optimization variables u_i!

↳ Example: $N = 2$ $\quad A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}$ $\quad B = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $\quad P = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ $\quad Q = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$ $\quad R = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Infinite Horizon

↳ We saw the J in receding horizon looks a lot like LQR

↳ If we remove the P-term, we exactly receive LQR again

↳ $J_\infty(x(0)) = \min_{u(\cdot)} \sum_{i=0}^{\infty} (x_i^T Q x_i + u_i^T R u_i)$

↳ The optimal input is then: $u^*(k) = -(B^T P_\infty B + R)^{-1} B^T P_\infty A x(k)$
where $P_\infty = A^T P_\infty A + Q - A^T P_\infty B (B^T P_\infty B + R)^{-1} B^T P_\infty A$
and $J_\infty(x(k)) = x(k)^T P_\infty x(k)$

↳ Advantage of Infinite Horizon ⇝ The optimal closed loop system with optimal input is asymptotically stable!

Exercise 4 **Infinite Horizon cost-to-go**

Consider a discrete-time autonomous system $x(k+1) = Ax(k)$ with the system matrix

$$A = \begin{bmatrix} 0.5 & 0 \\ 0.5 & 0.5 \end{bmatrix}.$$

Recall the definition of a quadratic cost function. Assume that our system is at state $x = [1; 0]$. We now want to compute the infinite horizon cost-to-go for the system with state cost matrix

$$Q = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}.$$

Complete the following tasks:

1) Verify that the given system is asymptotically stable.
2) Calculate the system states for $N = 3$ time steps and compute the value of the finite horizon cost function $J_0(x(0))$ with $x(0) = [1; 0]$.
3) Compute the infinite horizon cost-to-go using the discrete-time Lyapunov function. Compare this result to the result of the previous task.